

COGS 45 Week 1 Lab Assignment

Many potential types of **learning signals** are available to biological and artificial systems: for example, human learners may acquire information through **explicit teaching** from parents/siblings/instructors/peers, by **actively exploring our environment** to acquire particular pieces of information, receiving reward or punishment, or simply by **passive observation** of the world around us.

We explored a simple form of learning through “passive observation” in class — reconstructing the “sensory” (i.e., pixel-level) data the network receives via an auto-encoder.

Here, we further some basics of learning in feedforward neural networks, this time through explicit-teaching (i.e., “supervised” learning).

Feel free to work together on the modeling (or use an LLM), but please answer the conceptual questions on your own! (i.e., without using friends and without consulting LLMs).

Code to complete these exercises is available on Canvas.

(1) Feedforward neural networks, such as [multi-layer perceptrons \(MLPs\)](#), can learn to approximate any input/output function. But how well can they generalize to **held-out data** that the network has never seen before?

Train a feedforward neural network on only even digits from the MNIST dataset. Hold out a portion of even-digit examples from training to use as a test set. Then test on two types of held-out data:

- (a) Unseen examples of even digits (your held-out even-digit test set).
- (b) Odd digits the network has never seen before.

Plot training accuracy over epochs, report final test accuracy for both (a) and (b).

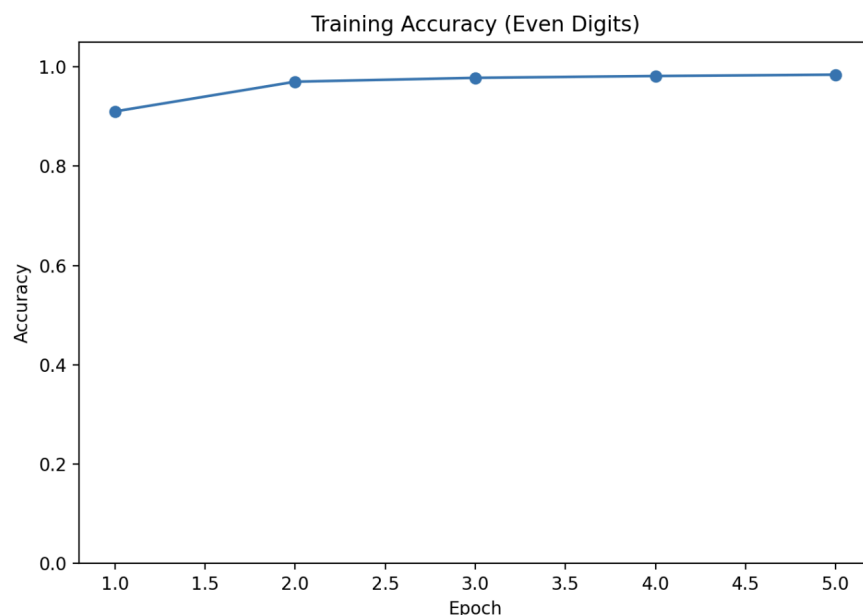


Figure 1

- a) Test accuracy on unseen EVEN digits: 0.9797 (almost 98%)
- b) Test accuracy on ODD digits: 0.0000 (0%)

We're feeding only even digits to the model and testing two things: whether it can recognize even digits it has not seen before (different handwritings of 0, 2, 4, 6, 8) and whether it can recognize odd digits it has never seen (1, 3, 5, 7, 9). The results indicate that the model can generalize to new examples of categories it has been trained on, and fails when it comes to generalize to entirely unknown categories. The model needs to learn representations for categories explicitly.

(2) Interpretation

A. In words, how well does the network perform in these two cases and WHY? Do the “confusion matrices” reveal anything about what features it has learned and/or what biases it has?

The network can predict unseen even digits within 98% accuracy because it can generalize what has been learned previously to new input through supervised learning. However, it performs poorly on odd digits, within 0% accuracy, because it has never seen those categories (3 and 7 have different shapes, for example), and it is forced to predict based on even digits. The “confusion matrices” reveal that the network cannot guess randomly, but it learns numbers based on visual features (e.g., curves, shapes) rather than abstraction. Therefore, there is a consistent mismatch between 3 and 8 because 3 looks like a half of 8 in the eyes of the model. This suggests that the network relies on the perceptual similarity rather than the abstract idea about numbers.

B. Do you think this network offers promise as a plausible model of how a human would learn to recognize handwritten numerical categories? Consider both the training signal (labeled examples) and the architecture.

Humans also learn from previous input, for instance, from teachers. We tend to group things based on how similar they are (individual digits are “numbers,” while letters constitute “words”). Similar to how neural networks learn, humans improve their learning by gaining more input through experience and formal instruction.

The difference is, humans only need so much experience to get better at identifying numbers. While we learn digits as a category, neural networks process them as pixels, which is a completely different type of information processing. Human brains learn digits using various tools, like conceptual versus procedural knowledge of numbers and continuous engagement with numeric math, while neural networks work in a feedforward way (input → hidden layers → output), meaning it follows only one direction. They need supervised learning, whereas humans use their lived experience, context, and reasoning to process the same information. Ultimately, they cannot count four apples in a basket because counting is beyond what they can perform.

(3) What are some relative advantages and disadvantages of supervised learning (like that used here) and self-supervised learning (e.g., autoencoders) for cognitive systems?

Supervised learning means cognitive systems learn from labels (the agent who's inputting the data). Advantages: (1) they perform well in tests (in the even digit example) and (2) that can be fast because they learn with what's provided in the past. Disadvantages: (1) Similar to LLMs, they need lots of data labeled for them, which is not how humans learn. (2) They cannot form new categories because they follow only one path. (3) They process numbers through pixels and heavily focus on visual features rather than the bigger picture. (4) When we learn, we start with foundational knowledge so we build up over time, whereas the neural networks have no idea about what they know and what they don't.

Self-supervised learning is when cognitive systems learn from the data on their own.

Advantages: (1) We don't need to label any data for these systems because they can learn from raw data. (2) They learn the general idea quickly because they look at patterns and similarities. Disadvantages: (1) They can divide numbers into categories but don't understand why it matters. 4 is a bunch of lines for neural networks, not anything meaningful. They have no understanding of numbers.

(4) These networks have been trained to map from images to number categories using the MNIST dataset.

A. Should we think of what the network has learned as representations of "numbers," like those of an adult human who has undergone formal education? Why or why not?

The way neural networks learn digits is through processing them as pixels (each and every corner), formulating shapes, and then making patterns. If we give a picture of something that looks similar to 8, they're still going to label it as a "number."

When we input 2, the network doesn't understand it is the addition of $1 + 1$. On the other hand, humans learn numbers in a much more meaningful way. Humans can operate math on them, which is a second step after learning numbers individually. In the example above, the network failed to identify odd digits as a broad categorization of numbers, while humans could still say they're numbers but just new.

B. How would we compare what a network has learned to what a human has learned? What kind of experiment or analysis might help us determine whether the network's internal representations are similar to a human's?

Similar to what we covered in Representational Similarity Analysis (RSA) in Week 2, we can ask the network to distinguish number vs. non-number categories (e.g., snake can look like 1).

OPTIONAL BONUS: How many labeled examples are required to successfully train a feedforward neural network using supervision, assuming the methods we use here? Train three supervised networks: one with 1 training example per category. One with 10 training examples per category. One with 1000 training examples per category. (this will require modifying the code). Plot the results on test data (held-out of training) using a single plot (matplotlib). Relate your results to the amount of data you might expect a human to require for the same task.